

```

1 // Problème 1
2
3 // Question 1 :
4 // Il faut que tout les clients soient parallélisés :
5 //     on télécharge chacun des morceaux de code en même temps
6 // Pas de parallélisation pour un serveur :
7 //     il ne pourra dans tous les cas fournir qu'une seule ressource
8 //     puisqu'il expose un socket sur un seul port
9
10 public class UsagerServer {
11     final static String hosts[] = {"host1", "host2", "host3"};
12     final static int nb = 3;
13     static String documents[] = new String[nb];
14
15     public static void main(String args[]) {
16         ServerSocket ss = new ServerSocket(8080);
17
18         while (true) {
19             Socket s = ss.accept();
20             OutputStream os = s.getOutputStream();
21             InputStream is = s.getInputStream();
22             ObjectOutputStream oos = new ObjectOutputStream(os);
23             ObjectInputStream ois = new ObjectInputStream(is);
24
25             int num = (int)ois.readObject();
26             oos.writeObject(documents[num]);
27         }
28     }
29 }
30
31 public class UsagerClient extends Thread {
32     final static String hosts[] = {"host1", "host2", "host3"};
33     final static int ports[] = {8081, 8082, 8083};
34     final static int nb = 3;
35     static String documents[] = new String[nb];
36
37     int target;
38
39     public UsagerClient(int i) {
40         target = i;
41     }
42
43     public void run() {
44         Socket s = new Socket(hosts[target], ports[target]);
45         OutputStream os = s.getOutputStream();
46         InputStream is = s.getInputStream();
47         ObjectOutputStream oos = new ObjectOutputStream(os);
48         ObjectInputStream ois = new ObjectInputStream(is);
49
50         oos.writeObject(target);
51         documents[target] = (String)ois.readObject();
52         s.close();
53     }
54
55     public static void main(String args[]) {
56         Thread th[] = new Thread[nb];

```

```

57
58         for (int i=0 ; i<nb ; i++) {
59             th[i] = new UsagerClient(i);
60             th[i].start();
61         }
62
63         for (int i=0 ; i<nb ; i++) {
64             th[i].join();
65         }
66     }
67 }
68
69
70
71 // Problème 2
72
73 // Ajouts pour la console en commentaire
74 // Visiblement on appelle ça une "inversion de contrôle"
75
76 public interface Daemon extends Remote {
77     public void exec(String cmd /* , Console console */) throws RemoteException;
78 }
79
80 public class DaemonImpl extends UnicastRemoteObject implements Daemon {
81     public DaemonImpl() {}
82
83     public void exec(String cmd /* , Console console */) throws RemoteException {
84         localExec(cmd /*, console */);
85     }
86
87     public static void main(String args[]) {
88         LocalRegistry.createRegistry(2000);
89         Naming.bind("//localhost:2000/daemon", new DaemonImpl());
90     }
91 }
92
93 public class RE {
94     public static void main(String args[]) {
95         // On met bien Daemon et pas DaemonImpl sinon on se prend -10pts
96         Daemon d = (Daemon)Naming.lookup("//"+args[0]+":2000/daemon");
97         d.exec(args[1] /*, new ConsoleImpl() */);
98     }
99 }
100
101 public interface Console extends Remote {
102     public void println(String s) throws RemoteException;
103 }
104
105 public class ConsoleImpl extends UnicastRemoteObject implements Console {
106     public Console() throws RemoteException {}
107
108     public void println(String s) throws RemoteException {
109         System.out.println(s);
110     }
111 }
112

```